# State licensing pipeline — handoff documentation

*Generated 2026-07-08 from repo commit `4bb85c01`. To get current numbers, run `python -m pipeline.client.handoff_doc` from the repo root (add `--pdf` for a PDF).*

**For engineers:** this document is the plain-English overview. The technical entry points are `CLAUDE.md` (repo root), `pipeline/client/README.md`, and `pipeline/config/README.md`.

## 1. What this is, and what you already have

This repository turns state licensing law into structured, comparable data: for each occupation and each state, what license (if any) is required, what it takes to get one, and what the license lets you do. The results are published as an interactive website.

The most important thing to know: **the raw law text is already in this repository.** Collecting statute text from the web is by far the most painful part of this work (see section 2), and we have already done it broadly. Think of the full scope as a grid: every occupation we track (94) crossed with every state (50) — 4,700 occupation/state cells in all. We've gotten law text saved for 3,202 of them (68%).

That is not the same as "100% coverage is easy and we're just behind" — finding the *right* law for a given occupation and state is often genuinely hard, not a checkbox. Some occupations have their own clearly-named chapter (easy); others are a handful of sections buried inside a much broader chapter that covers many unrelated things (hard to even locate, let alone confirm complete); a few states may not license an occupation at all, which looks identical to "not found yet" until someone checks.

**And it cuts both ways: even for the 68% we HAVE captured, we could have gotten it wrong without knowing it** — pulled a repealed statute, a different profession's act, or a chapter that no longer matches how the law is actually organized today. A confidently-processed record on the site is not a guarantee we found everything; it's our best current attempt, not a verified-complete one. Section 7 names the specific tools built to catch exactly this class of mistake and what they've actually caught so far — worth reading if "how much can we trust this" is the real question. Treat the 68% honestly: it's how far a real, ongoing effort has gotten, not a countdown to a known, verified finish line.

Day-to-day work — turning that saved text into published data — **never touches the web-scraping machinery.** You will not need to scrape anything unless you want brand-new material or a refresh of old material.

## 2. Data sources & integrations — the complete inventory

**Every occupation and state on the site traces back to exactly two external data sources.** That's the whole list — there is no third feed, API, or vendor anywhere in the pipeline. Both are read-only, free, and require no account or contract with either source's operator.

| Source | Coverage | Method | Code | Update cadence |
|---|---|---|---|---|
| **Justia** (law.justia.com) | 98.6% of corpora, 49/50 states | Scrape (real Chrome browser) | `pull_text.py` | Annual editions |

| | | | | |
|---|---|---|---|---|
| **PA Legislature** (legis.state.pa.us) | 1.4% of corpora, 1/50 states | Scrape (plain HTTP) | `pull_text.py` | At/near enactment |

*(Coverage measured from the saved files' own recorded URLs, not asserted. Both sources route through the same file, `pipeline/ingest/pull_text.py` — Justia via `fetch_via_browser`, PA via `is_pa_legis_url` and its section parser.)*

On update cadence, specifically:

- **Justia publishes each state's code as a year-labeled edition** (its own pages are literally titled things like "2024 Mississippi Code"). Several states we track (Kentucky, Oregon, Illinois, and others) publish *only* year-stamped pages — no undated "current" URL at all — and our pipeline has dedicated logic (`pull_text.py`'s `_dedupe_year_versions`) to detect this and always resolve to the latest year. That's direct, first-party evidence Justia refreshes annually, syncing to each state's newly codified text once that year's legislative session's changes are finalized — not a guess.
- **Pennsylvania's site is the state's own primary source, not a third-party aggregator**, so it isn't on a yearly sync cycle at all — a newly passed act should appear there at or near when it's officially published, well ahead of when it takes legal effect (Pennsylvania's default rule is 60 days after enactment unless the law says otherwise).
- Neither source updates in real time — the site always serves a snapshot from the last time someone ran a fetch, not a live lookup.

What needs a human, per source:

- **Justia** — a bot-detection challenge needs a **person** to open a real Chrome window and clear it once per session; it can reappear mid-run and pause until re-cleared. If Justia changes its page layout or URL scheme, the parser needs updating.
- **PA Legislature** — there's no Cloudflare-style bot wall here, so this source can be fetched by a plain automated script with nobody watching or clicking through anything. The only risk is the same as Justia's second case: if PA changes its site's URL scheme or page markup, the parser needs updating.

**A fetch is running right now** (checked live at generation time) — some of the 19 cells below are actively being pulled as this document was generated, so that count is falling.

Fetching is not fully autonomous — plan to run it yourself, and expect it to need attention. In practice: re-fetching or fetching new statute text isn't a cloud job you kick off and forget — it runs on your machine, it can take hours for a large batch, and the process itself stops on its own fairly often (a stalled request, a crash, the underlying browser session hiccupping) for reasons that aren't always obvious. A watchdog checks periodically and automatically restarts it when that happens, so it's more resilient than a plain script, but it is not "set and forget" — budget for someone to check on it and notice if it's stuck, not just kick it off and walk away for good.

One more related point: **finding the law is often harder than fetching it.** Many occupations' licensing rules aren't a standalone chapter — they're a few sections buried inside a much broader one (one insurance-agent type inside the entire insurance code). The repo includes automated locator tooling for this, and the per-state mappings it produces are saved in config, so that work doesn't have to be redone.

## 3. What's easy, what's cheap, and what's hard

The pipeline runs in stages, and they cost very different amounts of money and effort. Current state of the grid, as of generation:

| | Count | What it means |
|---|---|---|
| Published | 13 occupations | Site data built and ready to serve |
| Extracted, not published | 49 occupations | Structured data exists for at least some states; publishing is a free command |
| Ready to process | 2,167 occupation/state cells | Law text on disk; needs only the paid AI steps (~$0.20/cell, ~$424.10 for all of them) |
| New occupation, not set up yet | 285 cells (24 occupations) | Law text on disk, but this occupation has never been processed before — see below |
| Needs a fetch | 19 cells | Law located but text never pulled — requires the scraping setup |

The 13 published occupations: architecture, construction_trades, dentistry, dietetics, electrician, hvac, medicine, nursing, optometry, plumbing, podiatry, professional_engineering, public_accountancy.

In effort terms:

- **Free, minutes:** publish an occupation whose data is already extracted; re-run the deterministic cleanup step.
- **Cheap, no code:** run an already-fetched occupation/state through the two paid AI steps (about $0.20 per occupation/state cell). "Already fetched" means the raw statute text is saved in this repository — the AI steps just haven't been paid for yet. The dashboard (section 4) shows exactly which cells these are and quotes the cost before anything is spent.
- **Medium: bring a brand-new occupation online for the first time.** Before we can extract structured data for an occupation, an AI step has to figure out *what fields matter for it* — a nurse's data looks different from a plumber's (different requirements, different terminology) — and save that as a one-time template (called a "schema" in the code) for the occupation. This is a one-time step per occupation, not per state, and it's a small cost. This is the 285-cell row above: the law is already on disk, but nobody has designed that occupation's template yet. Once it exists, every state for that occupation becomes "ready to process" like any other. Some states may also still need a fetch.
- **Hard, human-supervised:** fetching new or updated statute text from the web (section 2). Plan for a person to be around while it runs.

## 4. The tools you'll actually use

What you need: Python (the repo ships a ready environment — see `pipeline/client/README.md`), an Anthropic API key for the two paid steps, and access to the site's hosting (Vercel) for publishing. Chrome is needed only for fetching.

Three commands cover everything except fetching (run from the repo root, using the project's Python environment):

1. **Status** — `python -m pipeline.client.status` — recomputes, for every occupation and state, what stage it's at, whether it's current or stale, and what it would cost to finish.
2. **Dashboard** — `python -m pipeline.client.dashboard` — builds an offline web page (`pipeline/results/status/dashboard.html`, open it in any browser) where you tick the occupations you want, see the total cost, and get the exact commands to run.
3. **Runner** — `python -m pipeline.client.run ...` — executes your picks: `clean` (free), `trim` and `extract` (the paid steps — each prints its cost estimate and waits for a typed "yes" before spending anything), `publish` (free).

Nothing in these tools fetches from the web or spends money on its own. The plain-English walkthrough, including re-fetching, is `docs/runbooks/client_refetch.md`.

## 5. What it costs — and what the costs depend on

Two stages call an AI model and cost money; everything else is free:

| Stage | Model used today | Est. cost |
|---|---|---|
| Trim — keep only the occupation's parts of the law | `claude-haiku-4-5-20251001` | ~$0.05 |
| Extract — turn trimmed text into structured data | `claude-sonnet-4-6` | ~$0.15 |
| Schema discovery — design an occupation's data template | `claude-sonnet-4-6` | small, one-time |

Three things to understand about these numbers:

- **They are estimates tied to today's models and prices.** The model names above were read from the code at generation time. Anthropic's per-token prices change, and bulk runs go through Anthropic's discounted batch API. Treat the per-cell figures as planning estimates, not invoices — the runner shows the estimate and asks before spending.
- **Swapping models is easy.** Each stage's model is a single setting (a `--model` flag / one constant per stage module). Moving to a cheaper, better, or different vendor's model is a small change; the prompts themselves live in a plain-text config file (`pipeline/config/prompts.yaml`), not in code.
- **The costs are per-run, not per-view.** The website serves previously-built data; people browsing the site cost nothing.

## 6. Everything adjustable lives in config files, not code

The pipeline's decisions — which occupations exist, how to recognize each one in a state's law, per-state quirks, what the AI is asked, rate limits — are all plain-text YAML files in `pipeline/config/`, each documented in `pipeline/config/README.md`:

- `professions.yaml` — the occupation list and the patterns used to find each one.
- `state_structure.yaml` — per-state source and layout overrides (the single source of truth for "where does this state keep this law").

- `prompts.yaml` — the instructions given to the AI at each paid step.
- `limits.yaml` — every numeric knob (fetch delays, caps, thresholds).

A non-programmer can read these; a technical person can change them without touching Python.

## 7. How much to trust the data

- **It's AI-extracted, with checks.** Structured records are extracted by an AI model from statute text, then run through automated consistency checks (`pipeline/eval/check_consistency.py`). It can still be wrong; every record links back to its source law so a human can verify.
- **"Done" doesn't mean "done forever."** Each cell's output depends on its inputs (the law text) and its recipe (prompt + schema + model + code). The repo stamps every output and can tell you whether it's *current*, *stale* (an input or the recipe changed since it was built), or *unstamped* (built before tracking existed). Right now: 750 extracted cells — 350 current, 0 stale, 400 unstamped. The dashboard color-codes this.
- **Some blanks are deliberate.** A hand-verified list (`pipeline/config/verified_absences.yaml`) records occupation/state pairs where the state genuinely has no license — shown as "not state-regulated (verified)," distinct from "we haven't processed this yet."
- **Having a chapter isn't the same as having every relevant law.** Some states put discipline, fees, or hearing rules in shared "umbrella" chapters that apply to many occupations; the repo detects and pulls these, but completeness auditing is ongoing.
- **We actively hunt for our own mistakes, and they do happen.** Two purpose-built audits exist because processed-and-published is not the same as verified-correct: `pipeline/discover/audit_corpus.py` re-checks whether a pulled corpus is actually that occupation's licensure law — in its validated run across 4 trades / 146 states, it correctly flagged ~7 real mistakes (a repealed statute counted as current law, a fire-safety or criminal statute pulled instead of the licensing act) with zero false alarms. `pipeline/eval/corpus_drift.py` catches a different failure: a corpus that silently stopped matching its own mapping after the mapping changed (a real bug it found: a state's committed corpus was still answering to a mapping that had been replaced, and nobody had noticed because the page still "had text" on it). Both exist precisely because a wrong-but-confident-looking record is possible — running them, and reading what they flag, is the honest way to answer "how much can we trust this," not the 350-current/0-stale/400-unstamped counts above on their own.

---

*Regenerate with `python -m pipeline.client.handoff_doc [--pdf]`.*